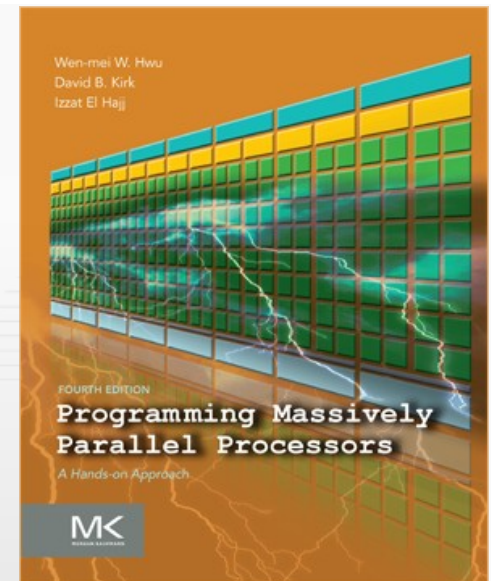
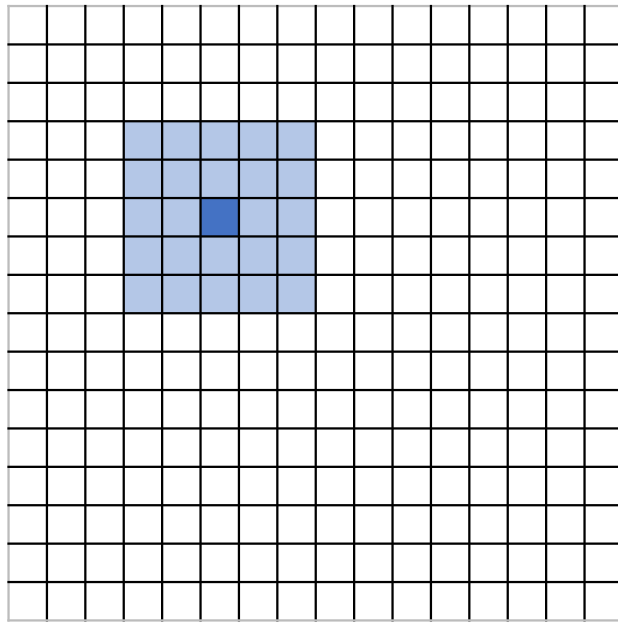


Programming Massively Parallel Processors

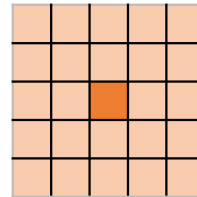
A Hands-on Approach

CHAPTER 7 > Convolution

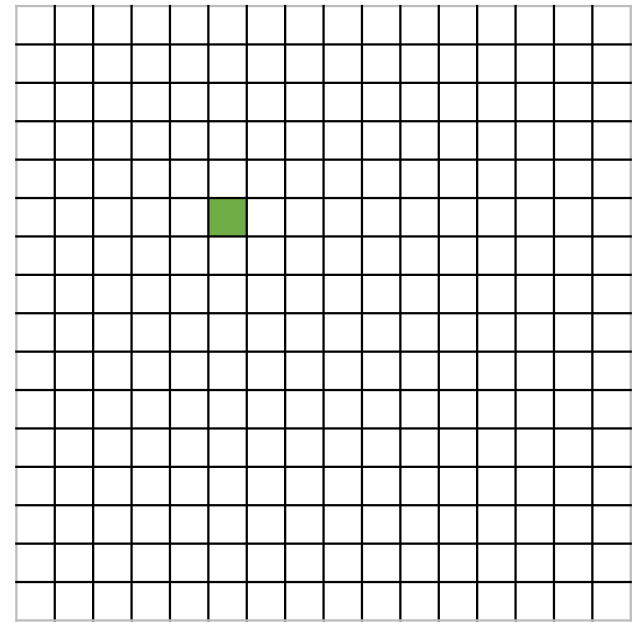




input



filter



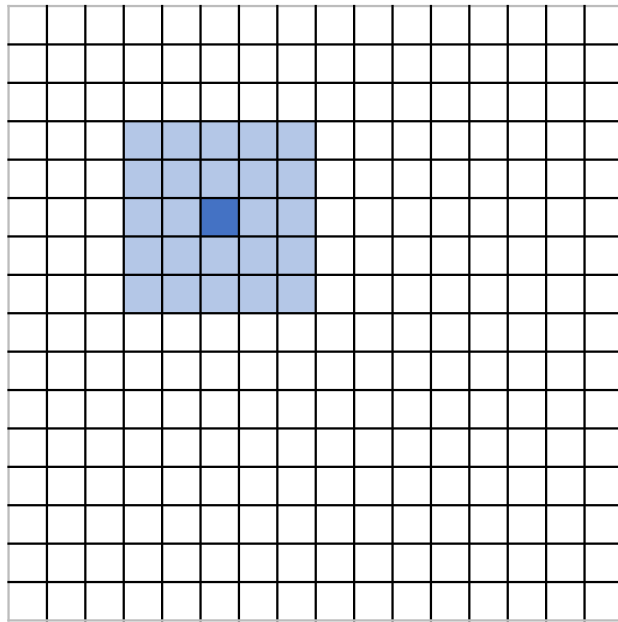
output

Every **output element** is a weighted sum of the neighboring **input elements**

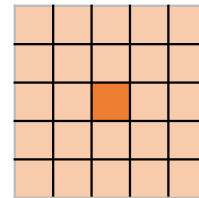
Image blur seen before was a special case where all weights are the same

In general, weights are determined by a convolution **filter**

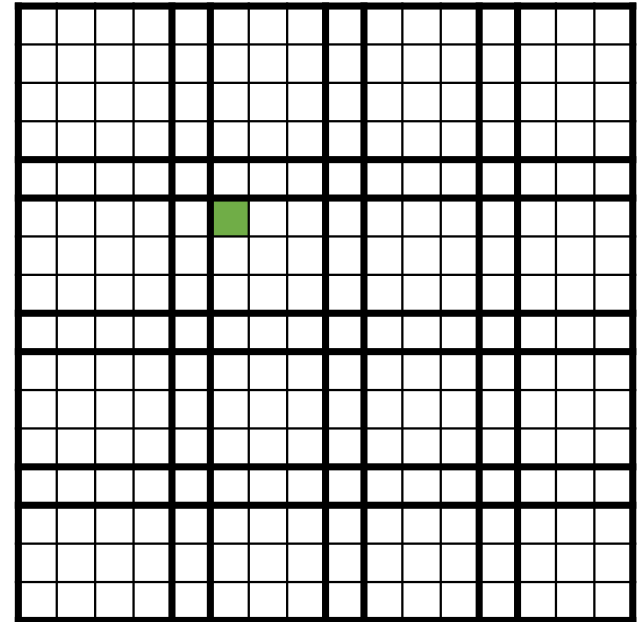
- Commonly used in signal processing, image processing, video processing, etc.
- Usually used to transform signals or pixels to more desirable values
 - e.g., Gaussian blur, sharpen, edge detection, etc.
 - Transformation depends on the weights in the filter
- Using 2D as an example, but can also be 1D or 3D



input



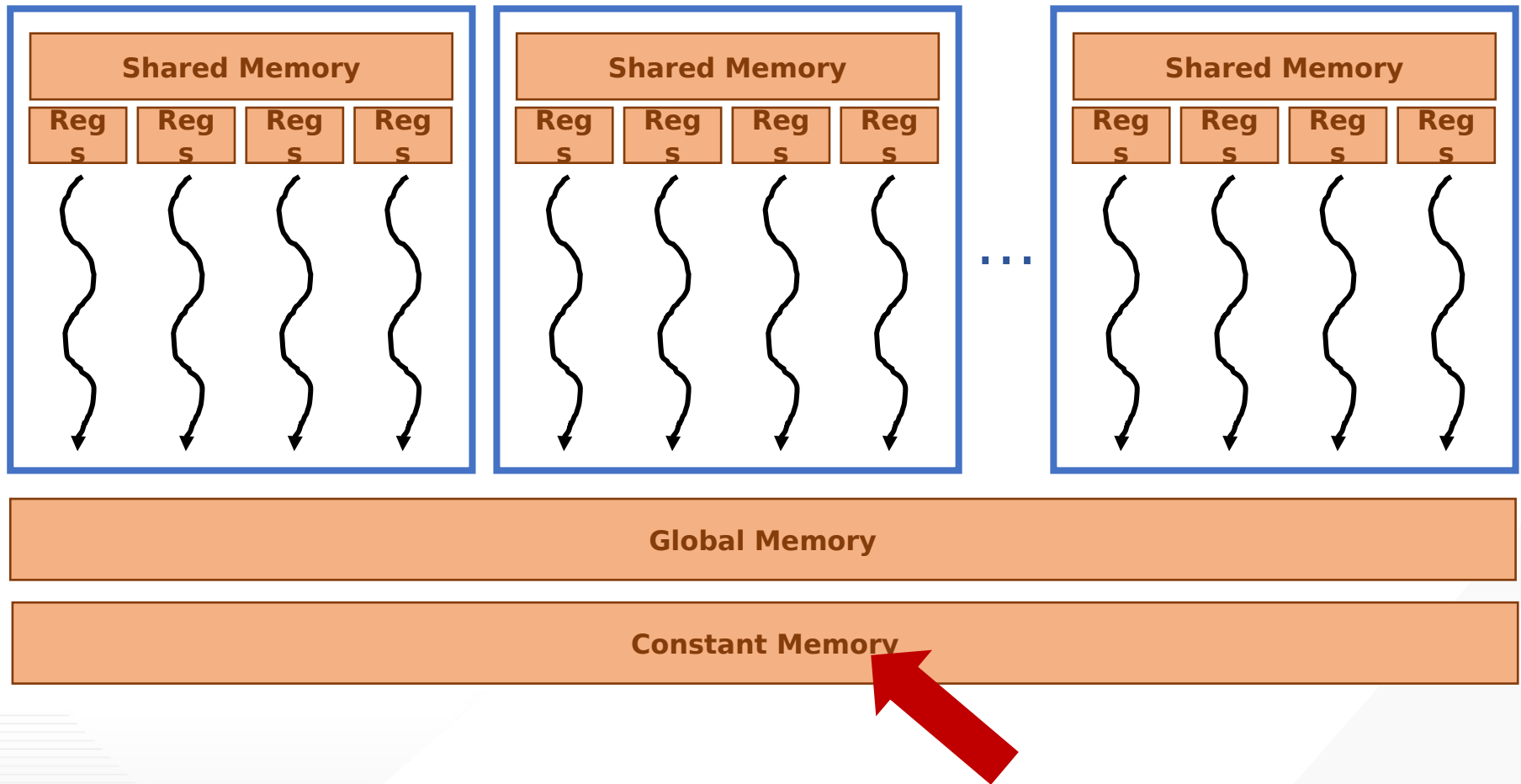
filter



output

Parallelization approach: Assign one thread to compute each **output element** by looping over **input elements** and **filter** weights

- Observations:
 - The filter is typically small
 - The filter is constant (weights do not change)
 - The filter is accessed by all threads in the grid
- Optimization: store the filter in **constant memory** for quicker access



- Declare constant memory array as global variable

```
__constant__ float filter_c[FILTER_DIM][FILTER_DIM];
```

- Must initialize constant memory from the host:

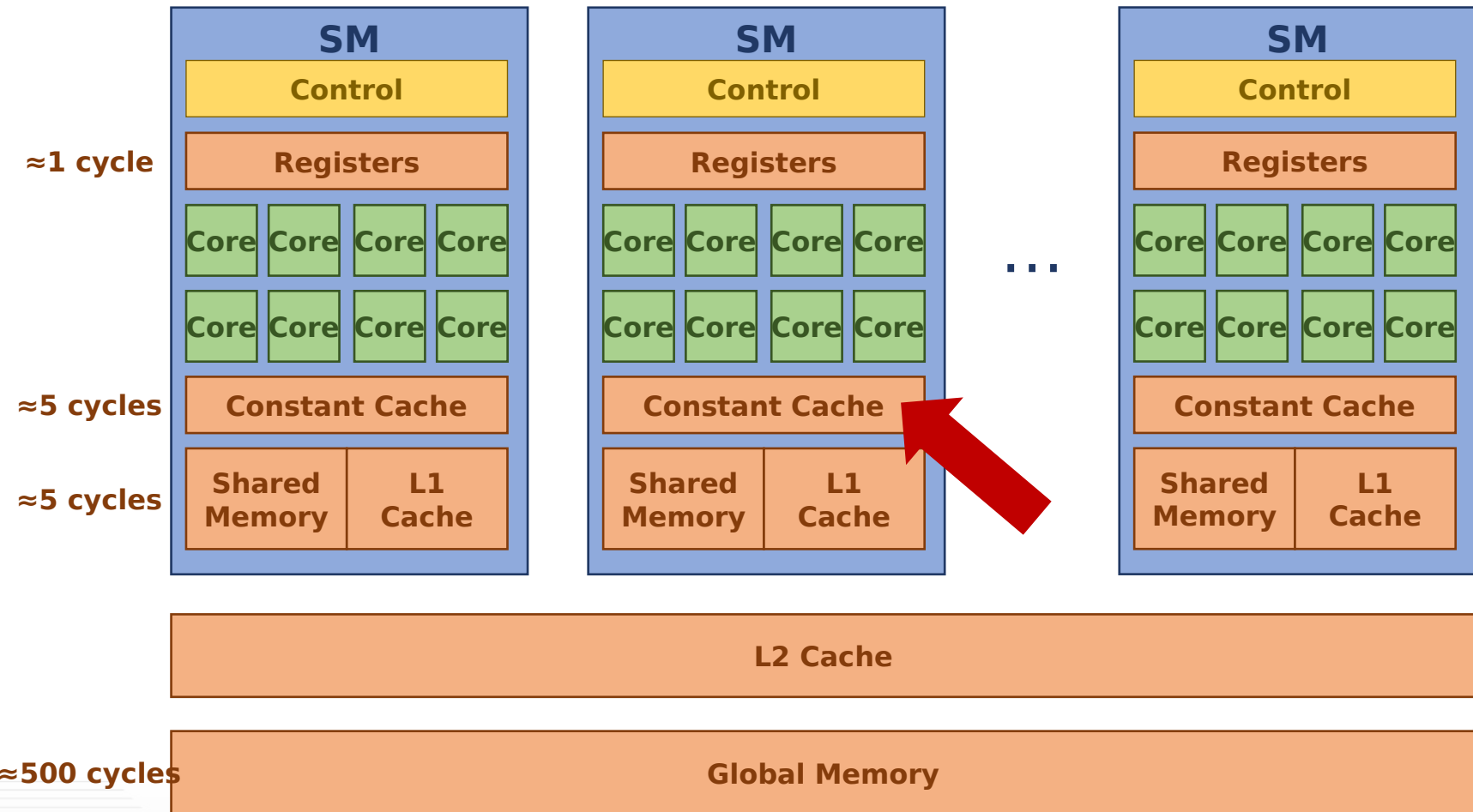
- Cannot modify during execution

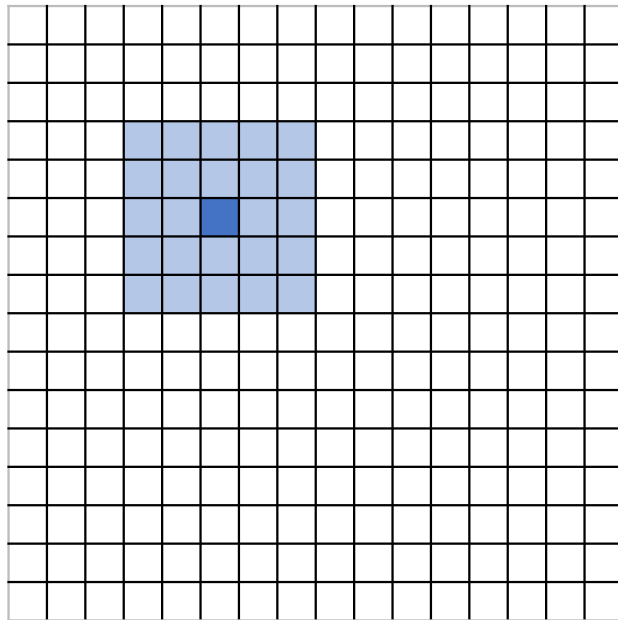
```
cudaMemcpyToSymbol(filter_c, filter, FILTER_DIM*FILTER_DIM*sizeof(float));
```

- Can only allocate up to 64KB
 - Otherwise, input is also constant, but it is too large to put in constant memory

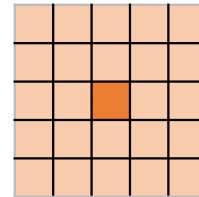
- Constant data: easier to build an efficient cache
 - No need to support write back
 - No need to support coherence
- Small size: minimize evictions
 - Cache for constant memory has low miss rate

Recall: Memory in the GPU Architecture

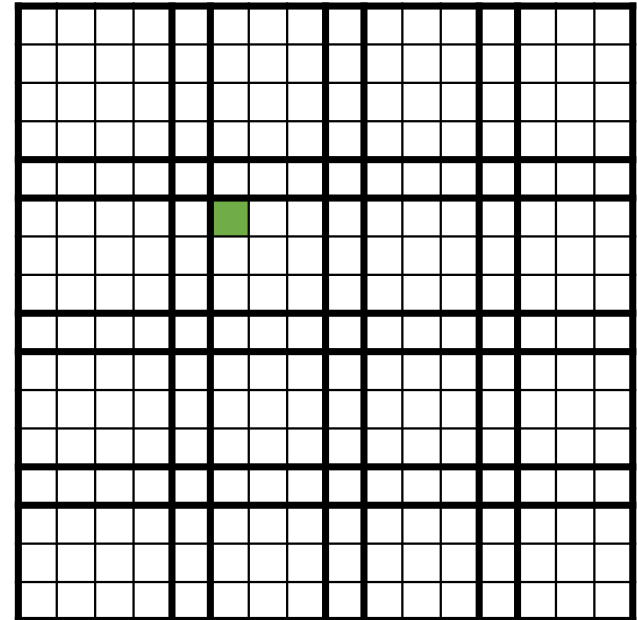




input
(in global
memory)



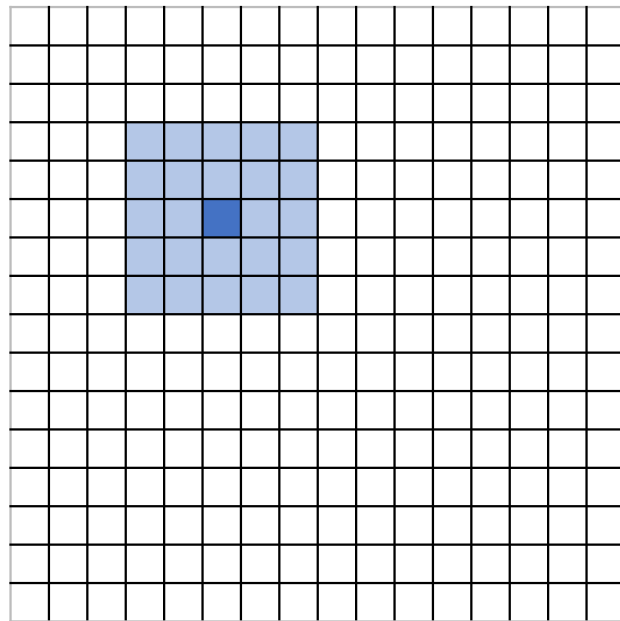
filter
(in
constant
memory)



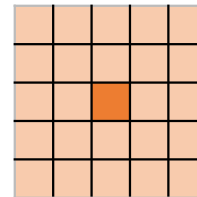
output

Parallelization approach: Assign one thread to compute each **output element** by looping over **input elements** and **filter** weights

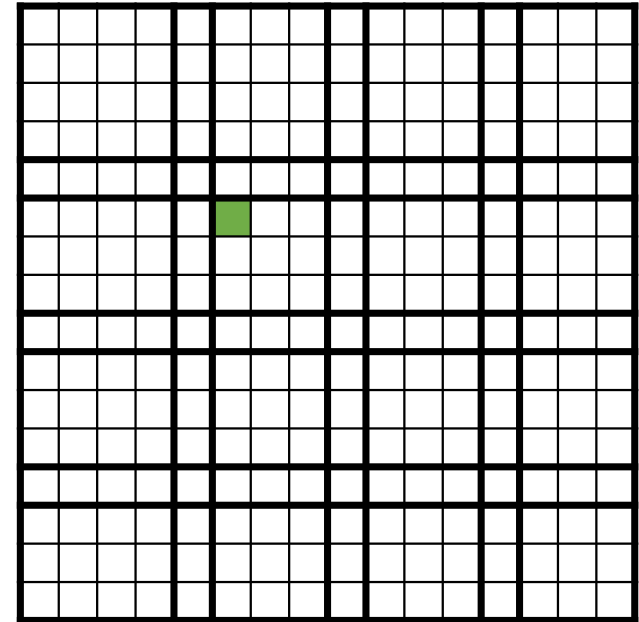
```
__global__ void convolution_kernel(float* input, float* output, unsigned int width,
                                unsigned int height) {
    int outRow = blockIdx.y*blockDim.y + threadIdx.y;
    int outCol = blockIdx.x*blockDim.x + threadIdx.x;
    if (outRow < height && outCol < width) {
        float sum = 0.0f;
        for(int filterRow = 0; filterRow < FILTER_DIM; ++filterRow) {
            for(int filterCol = 0; filterCol < FILTER_DIM; ++filterCol) {
                int inRow = outRow - FILTER_RADIUS + filterRow;
                int inCol = outCol - FILTER_RADIUS + filterCol;
                if(inRow >= 0 && inRow < height && inCol >= 0 && inCol < width) {
                    sum += filter_c[filterRow][filterCol]*input[inRow*width +
inCol];
                }
            }
        }
        output[outRow*width + outCol] = sum;
    }
}
```



input
(in global
memory)

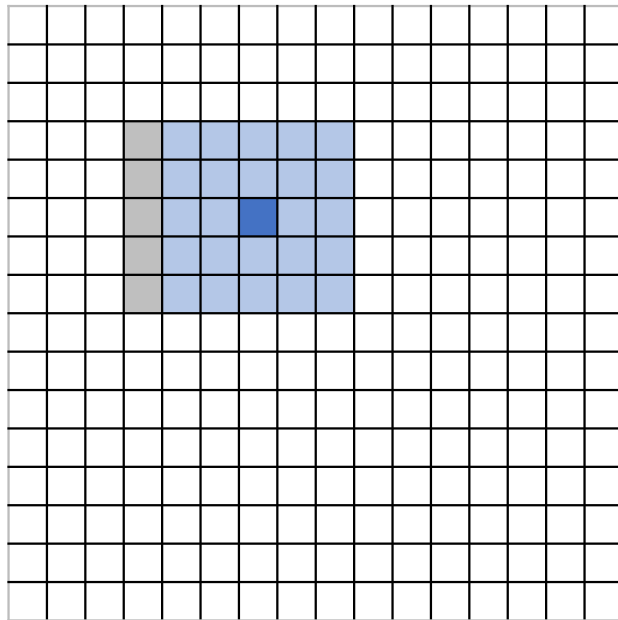


filter
(in
constant
memory)

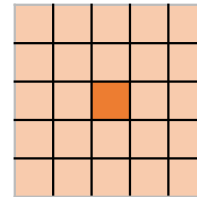


output

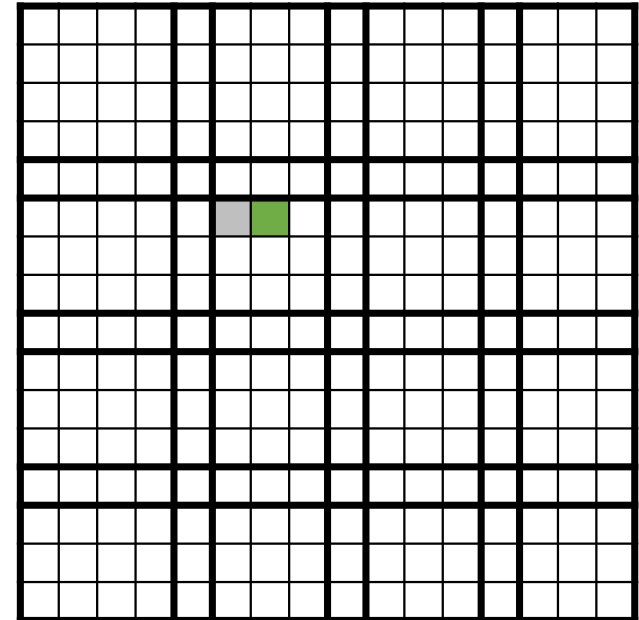
Observation: Threads in the same block load some of the same input elements



input
(in global
memory)

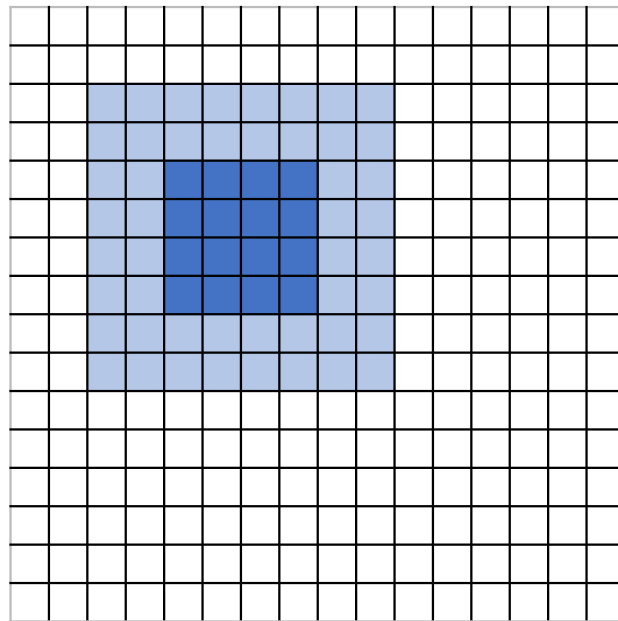


filter
(in
constant
memory)

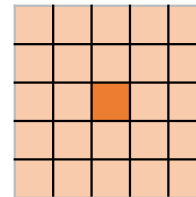


output

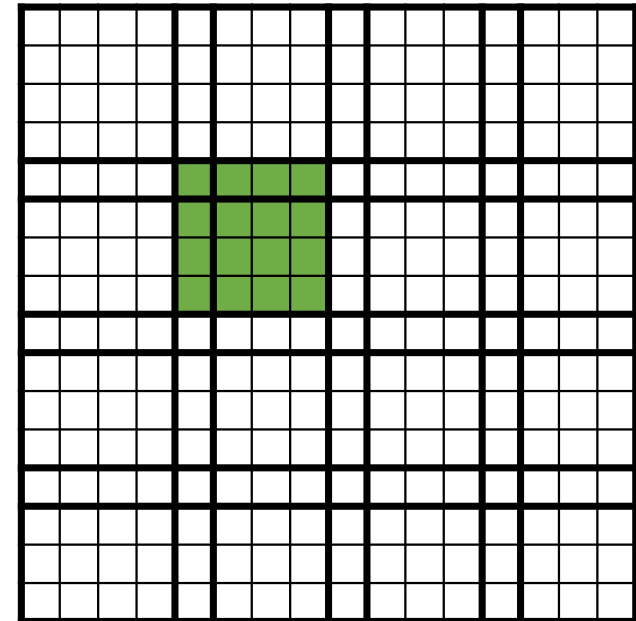
Observation: Threads in the same block load some of the same input elements



input
(in global
memory)



filter
(in
constant
memory)

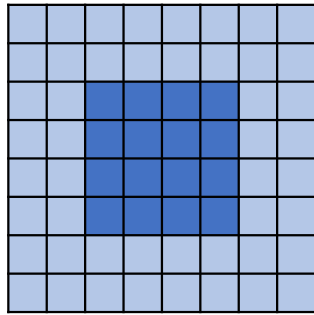


output

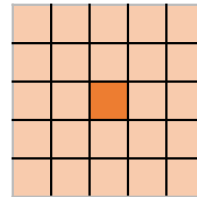
Observation: Threads in the same block load some of the same input elements

Optimization: Each thread loads one input element to shared memory and other threads access the element from shared

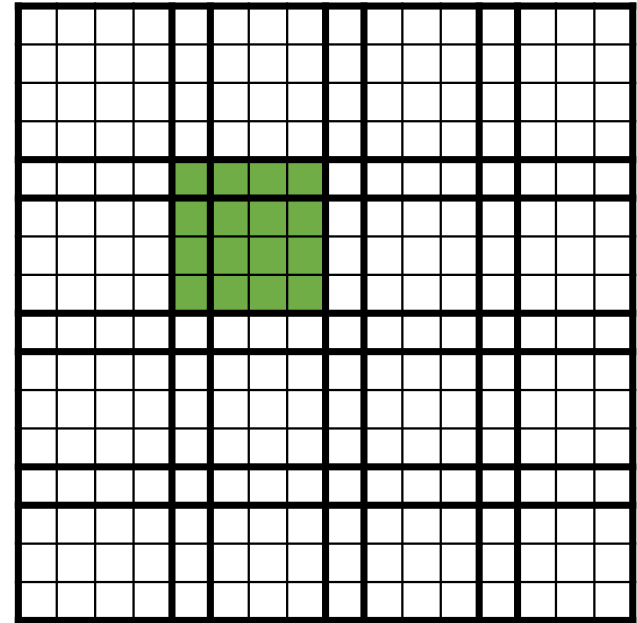
memory



input_{tile}
(in shared
memory)

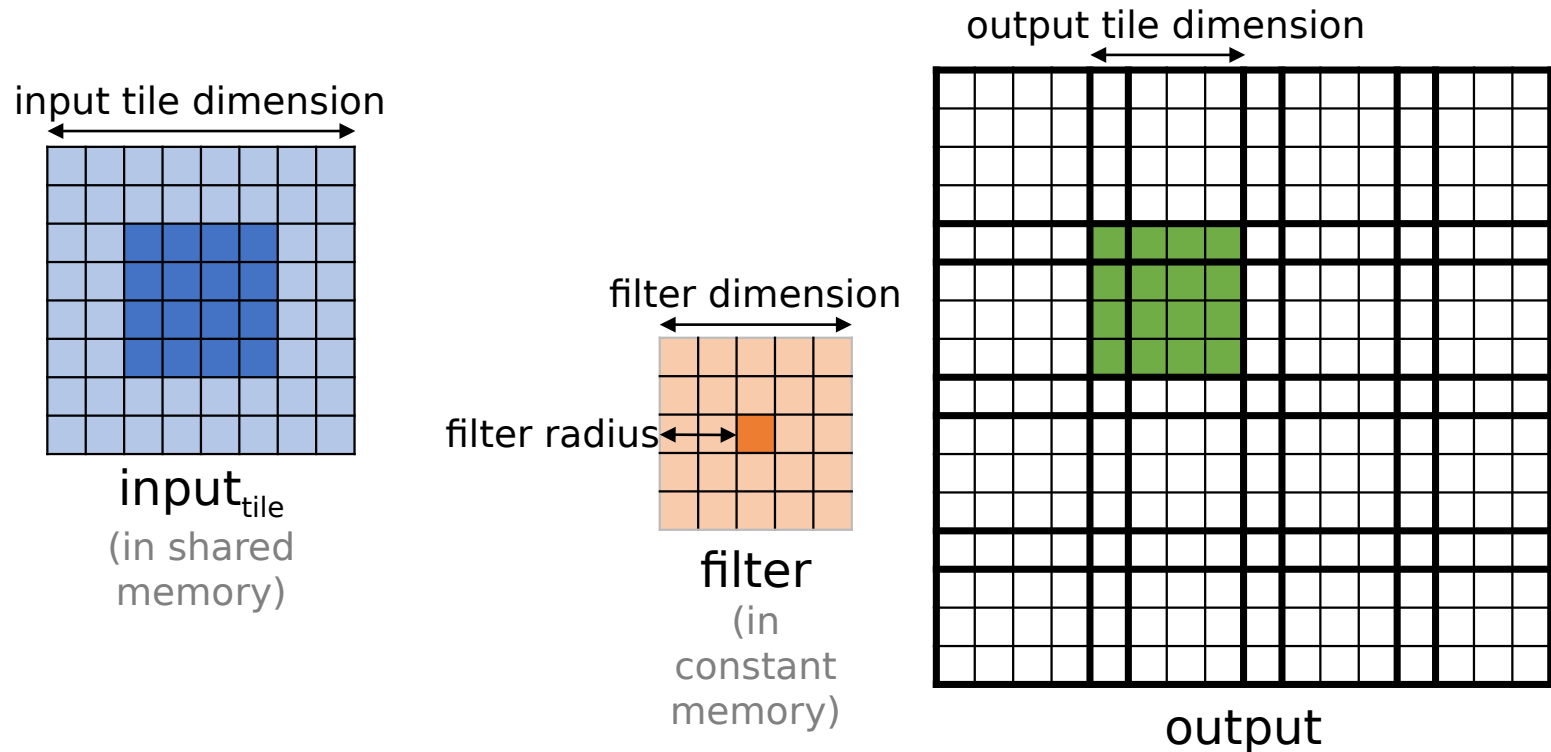


filter
(in
constant
memory)



output

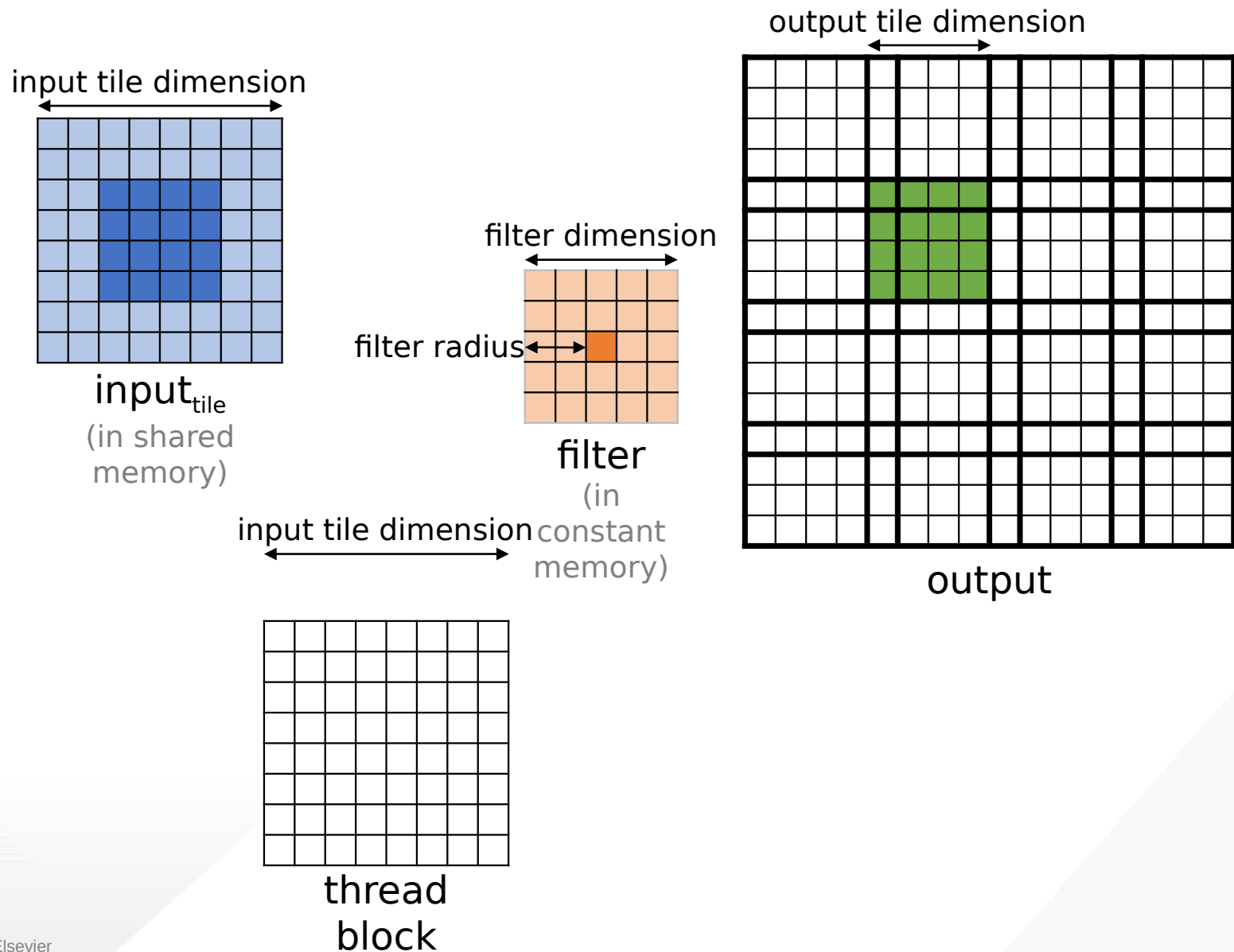
Optimization: Each thread loads one input element to shared memory and other threads access the element from shared memory

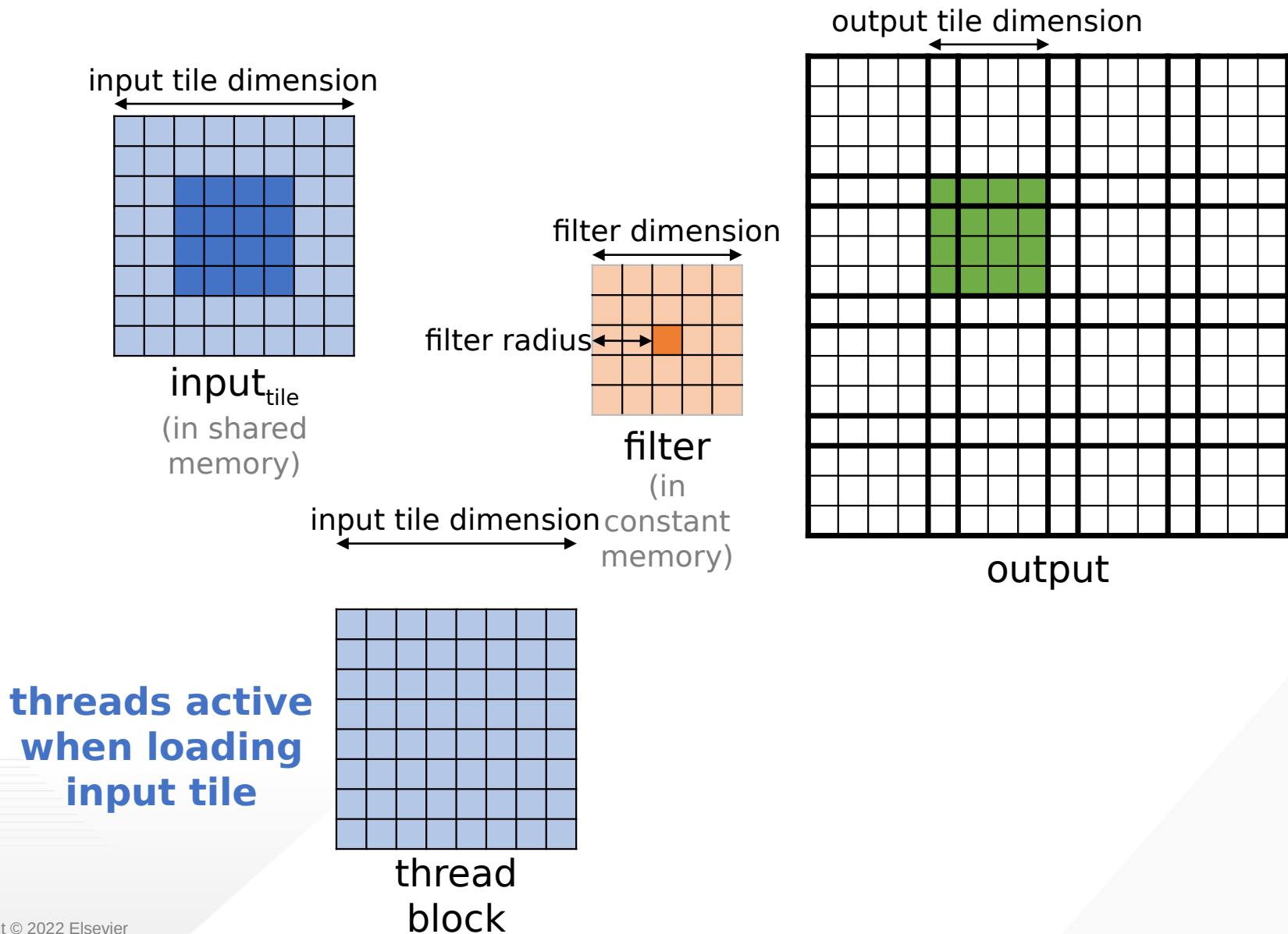


Challenge: Input and output tiles have different dimensions

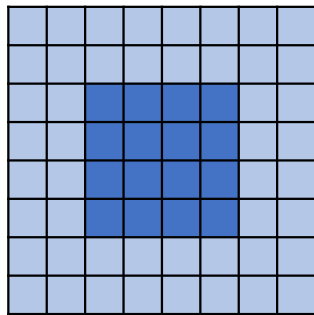
(input tile dimension = output tile dimension + 2 × filter radius)

Solution: Launch enough threads per block to load the input tile to shared memory, then use a subset of them to compute and store the output tile



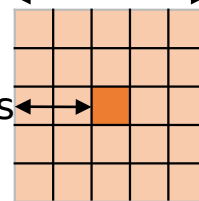


input tile dimension



input_{tile}
(in shared
memory)

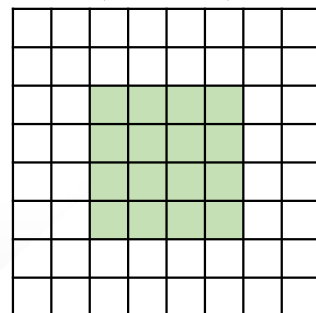
filter dimension



filter radius

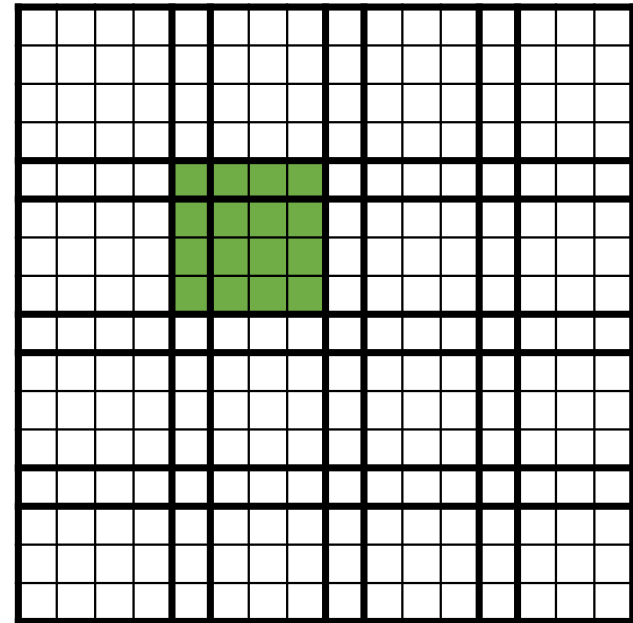
filter
(in
constant
memory)

input tile dimension
output tile dimension



thread
block

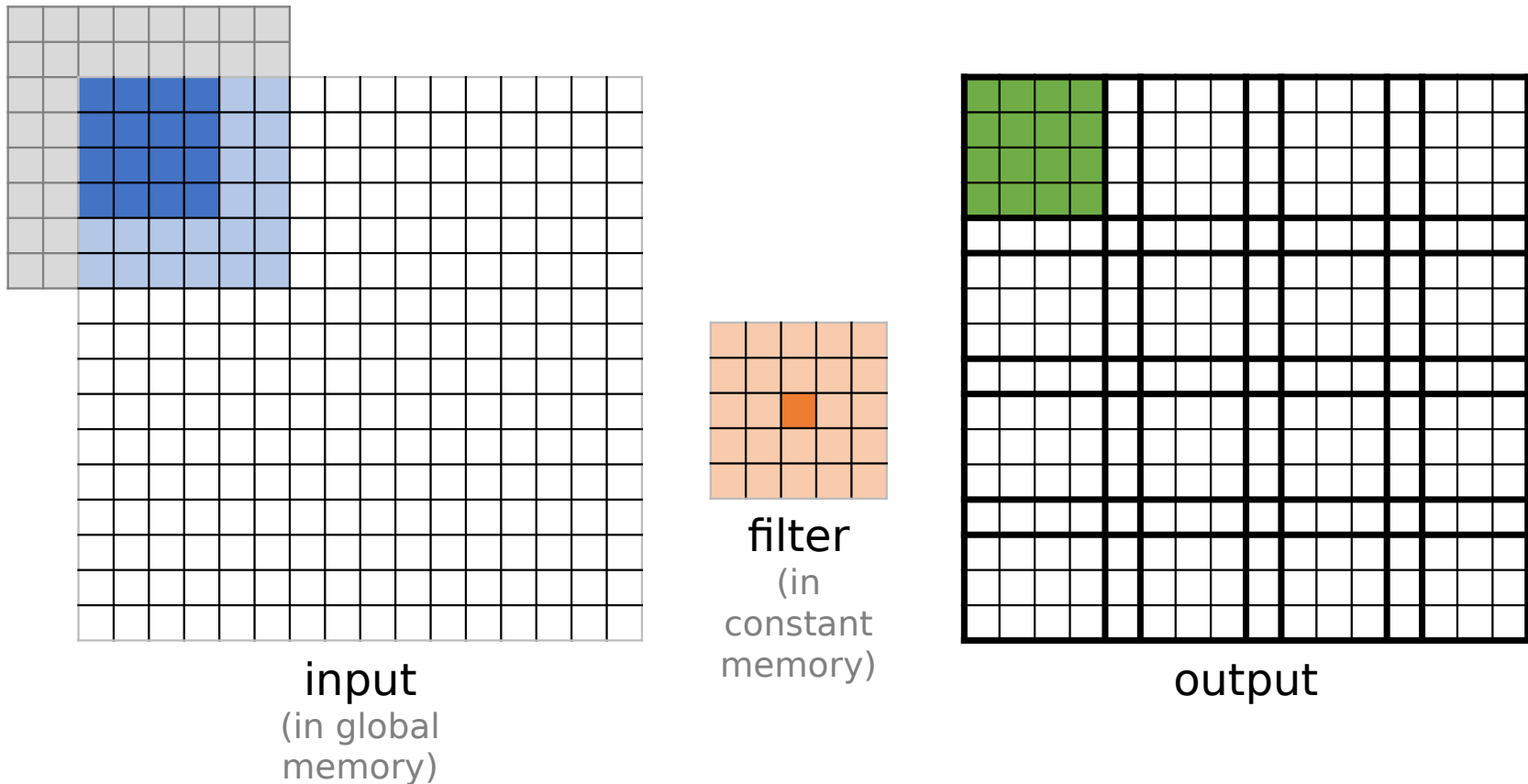
output tile dimension



output

**threads active
when computing
and storing the
output tile**

- Considering an $M \times M$ filter
- Without tiling:
 - Operations per thread: M^2 adds + M^2 muls = $2M^2$ OP
 - Global loads per thread: $M^2 \times 4 B = 4M^2 B$
 - Ratio: $(2M^2 \text{ OP}) / (4M^2 B) = 0.5 \text{ OP/B}$
- With tiling:
 - Considering tile dimensions: input = T , output = $T-M+1$
 - Operations per block: $(T-M+1)^2 \times 2M^2 \text{ OP}$
 - Global loads per block: $T^2 \times 4 B$
 - Ratio: $((T-M+1)^2 \times 2M^2 \text{ OP}) / (T^2 \times 4 B) = 0.5M^2(1 - (M-1)/T)^2$
 - For $M=5$ and $T=32$: 9.57 OP/B ($\approx 19\times$ improvement!)



Threads computing output elements at the boundary access input elements that are out of bounds (also called *ghost* elements)

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0						
0	0						
0	0						
0	0						
0	0						
0	0						

input_{tile}
(in shared
memory)

filter
(in
constant
memory)

output

Threads computing output elements at the boundary access input elements that are out of bounds (also called *ghost* elements)

Solution: Store zero to shared memory tile for our of bounds input elements

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.